TCP Explained

ACKs, flow control, congestion control, head-of-line blocking

Watch the 2-min video

> **In one line:** TCP turns unreliable IP into a reliable, ordered byte stream using sequence numbers, ACKs and retransmission, adds flow and congestion control so it doesn't drown the receiver or the network, and pays for strict ordering with head-of-line blocking.

1 A reliable stream on an unreliable network

IP is best effort. Packets can be lost, duplicated, or arrive out of order. Yet your database results always arrive complete and in order. That is TCP's job.

TCP gives the application a **reliable, ordered byte stream**. The sender splits the stream into **segments** and gives every byte a **sequence number**. The receiver uses those numbers to put segments back in order and to drop duplicates. Your code just reads bytes; it never sees packets.

2 ACKs and retransmission

The receiver sends **acknowledgements**. An ACK is *cumulative*: "ACK 1001" means "I have every byte up to 1000; send me byte 1001 next." If a segment is lost, later segments still arrive, but the receiver can only repeat the same ACK. The sender retransmits the missing segment in one of two ways:

- **Timeout**: no ACK arrived within the retransmission timeout (RTO), which is computed from measured round-trip times (at least 200 ms on Linux, often around 1 s on long paths).
- **Fast retransmit**: three **duplicate ACKs** tell the sender something is missing while later data is still getting through, so it resends right away.

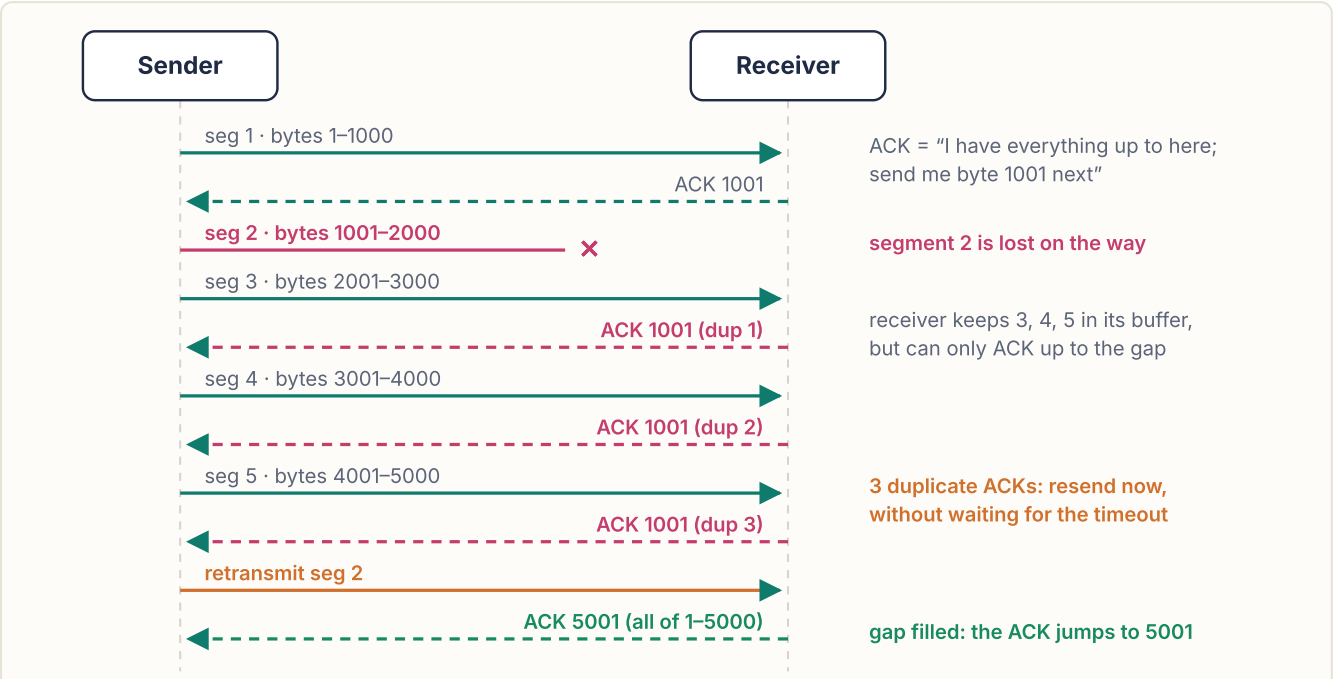


Figure 1. Segment 2 is lost. Segments 3, 4 and 5 each trigger a duplicate "ACK 1001"; on the third, the sender resends segment 2, and the ACK jumps to 5001.

3 Flow control: protect the receiver

Every ACK carries a **receive window**: how much free buffer space the receiver has. The sender may never have more unacknowledged data in flight than that. If the receiving app reads slowly, its buffer fills, the window shrinks and the sender must slow down. If the window reaches **zero**, the sender pauses and probes until the window opens again.

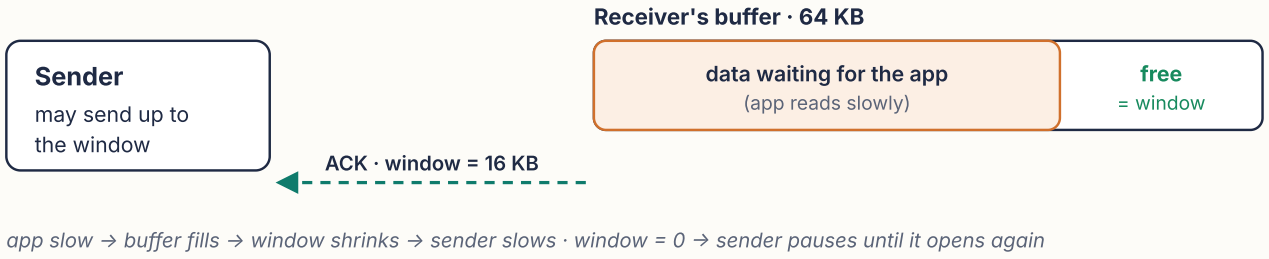


Figure 2. The receive window is the free part of the receiver's buffer. A slow consumer shrinks it, and that slows the sender.

4 Congestion control: protect the network

Flow control knows nothing about the routers in between. **Congestion control** does: the sender keeps a **congestion window** and sends no more than the smaller of the two windows.

- **Slow start.** A new connection starts small (Linux: 10 segments, about 14 KB) and roughly **doubles every round trip**.
- **Back off on loss.** When it detects loss, it cuts its window, then grows again more carefully.
- **CUBIC**, the Linux default, treats packet loss as the signal and cuts to about 70% on each loss. **BBR**, from Google, instead estimates the path's bandwidth and minimum round-trip time and paces sending to match, so random loss doesn't make it back off.

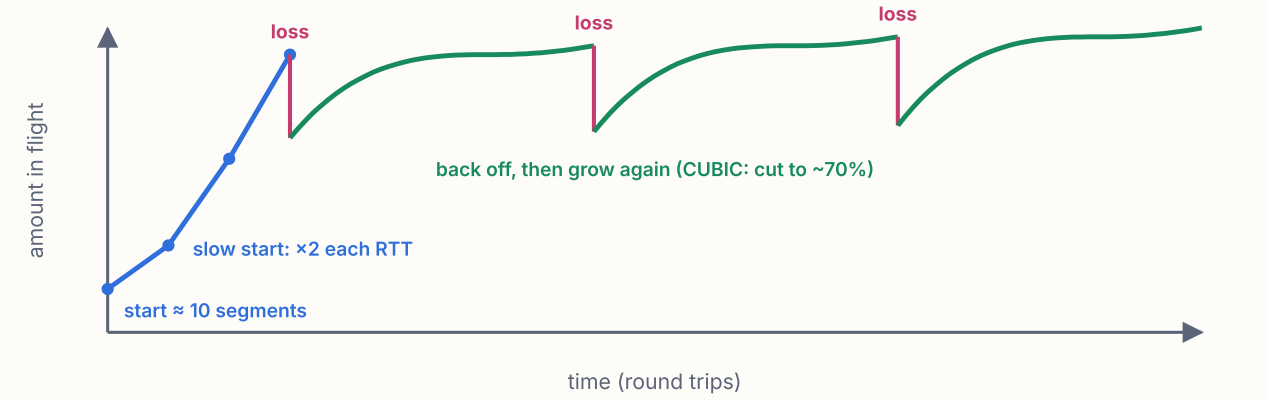


Figure 3. Slow start doubles the window every round trip; each loss cuts it, and it grows back. A short request may finish before slow start ever reaches full speed.

WORKED EXAMPLE: WHY A NEW CONNECTION IS SLOW FOR BIG RESPONSES

A 1 MB response on a fresh connection, 100 ms RTT, starting at 10 segments (~14 KB). Round trips carry about 14, 28, 56, 112, 224, 448 KB: after 6 round trips about 880 KB has been sent, so the 1 MB finishes in the 7th. That is roughly **0.7 s** of transfer time, even on a fast link, plus the handshake. On a warm, reused connection whose window is already large, the same response can take one or two round trips.

5 The price of order: head-of-line blocking

Strict ordering has a cost. Say segment 2 is lost, but 3 and 4 have already arrived. The app can't read them yet: TCP only hands over bytes in order. Everything waits until segment 2 is retransmitted, at least one more round trip. **One lost packet stalls the whole stream.**



TCP delivers bytes strictly in order, so one lost segment stalls everything behind it, even data already received.

Figure 4. Head-of-line blocking: segments 3 and 4 sit in the buffer, ready but unreadable, until segment 2 is resent.

Where you meet it	What happens
HTTP/1.1	One request at a time per connection, so browsers open about 6 connections per host in parallel.
HTTP/2	Many streams multiplexed over one TCP connection. Fixes HTTP-level blocking, but one lost packet still stalls every stream.
HTTP/3	Runs on **QUIC over UDP**, which orders each stream separately, so a loss only stalls its own stream.
Database drivers	Keep connection pools, so queries reuse open, warmed-up connections instead of paying handshakes and slow start.

COMMON MISTAKES

- Mixing up flow control (protects the receiver, receive window) and congestion control (protects the network, congestion window).
- Thinking TCP preserves message boundaries. It is a byte stream; your protocol must frame messages (length prefix, delimiter).
- Assuming HTTP/2 removes head-of-line blocking entirely. It removes it at the HTTP layer, not at the TCP layer.
- Benchmarking with fresh connections and blaming the server for slow-start time.

6 Key terms

Segment: one TCP chunk of the byte stream

Sequence number: position of a byte in the stream

ACK: next byte expected (cumulative)

Fast retransmit: resend after 3 duplicate ACKs

RTO: retransmission timeout

Receive window: receiver's free buffer: flow control

Congestion window: sender's estimate of what the network can take

Slow start: window doubles each RTT from a small start

HOL blocking: one loss holds back later data

7 Interview questions

Q1. How does TCP guarantee delivery? BASIC

Sequence numbers on every byte, cumulative ACKs from the receiver, and retransmission of anything not acknowledged (after a timeout or 3 duplicate ACKs). Flow and congestion control stop it from overwhelming the receiver or the network.

Q2. What is head-of-line blocking? BASIC

TCP delivers bytes strictly in order, so one lost segment holds back everything behind it, even data that has already arrived. The app waits at least one extra round trip for the retransmission.

Q3. Flow control vs congestion control? BASIC

Flow control protects the receiver: the receive window in each ACK says how much buffer is free. Congestion control protects the network: the sender's congestion window grows with slow start and shrinks on loss. The sender uses the smaller of the two.

Q4. A consumer service is slow and the producer's TCP sends stall, though the network is idle. Why? SCENARIO

Flow control. The consumer's receive buffer is full because the app isn't reading, so it advertises a zero window and the sender pauses. Fix the slow reader (more workers, async processing), not the network.

Q5. Why do database drivers and HTTP clients use connection pools? SCENARIO

A new connection pays a handshake (1 RTT, plus TLS) and starts in slow start at about 14 KB per round trip. A pooled connection is already open and its congestion window has grown, so queries start immediately.

Q6. Your HTTP/2 API is slow for mobile users on lossy networks. What could help? SCENARIO

All streams share one TCP connection, so each lost packet stalls every request. HTTP/3 (QUIC over UDP) recovers per stream, and BBR copes better with random loss than loss-based CUBIC.

Q7. Why retransmit after three duplicate ACKs instead of one? TRICKY

Packets can be reordered, which also produces a duplicate ACK or two. Three duplicates are strong evidence of a real loss while later data still flows, so the sender resends without waiting for the much longer timeout.

Q8. You send two 100-byte messages over TCP. The receiver's read() returns 200 bytes at once. Bug? TRICKY

No. TCP is a byte stream with no message boundaries; reads can merge or split writes. The application protocol must frame messages, for example with a length prefix, as HTTP/2, gRPC and database protocols do.

Next: SD-B-004 · TCP Handshake, FIN/RST & TIME_WAIT backendroadmap.com